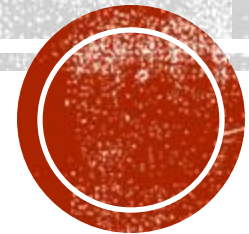


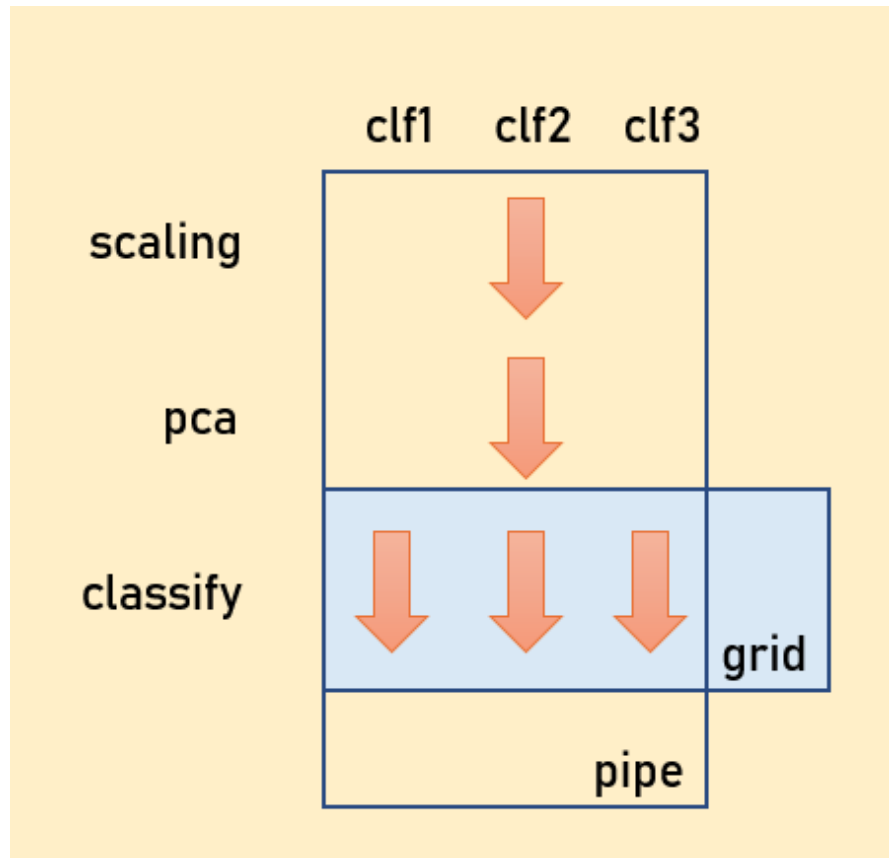
WEEK 7

Applied Pattern Recognition: Pipelines and Reliable Evaluation



REAL WORLD PROCESS

The whole pipeline



Data preparation/ Pre- processing



Feature Extraction



Classification



Evaluation (Understand/ Choose the best)



PIPELINE EXAMPLE

Preparation

- Fetch images
- Split the data

Feature Extraction

- Choose the parameter (.97 eigen value)
- PCA

Classify

- Train the image
- Make prediction

Evaluate

- Evaluate the result



PIPELINE

- Modify the code into pipeline

```
from sklearn.pipeline import make_pipeline
pipe = make_pipeline(PCA(.97), LogisticRegression())
pipe.fit(X_train, y_train)
print("Test acc:", pipe.score(X_test, y_test))

y_pred = pipe.predict(X_test)
print(classification_report(y_test, y_pred, target_names=target_names))
```



PIPELINE WITH STEPS

```
# Make sure you include all libraries.
```

```
steps = [ ('Dim' , PCA(n_components=150)) ,  
          ('classify' , LogisticRegression()) ]
```

```
# define the pipeline object.  
from sklearn.pipeline import Pipeline
```

```
pipeline = Pipeline(steps)  
pipe.fit(X_train, y_train)  
y_pred = pipe.predict(X_test)
```

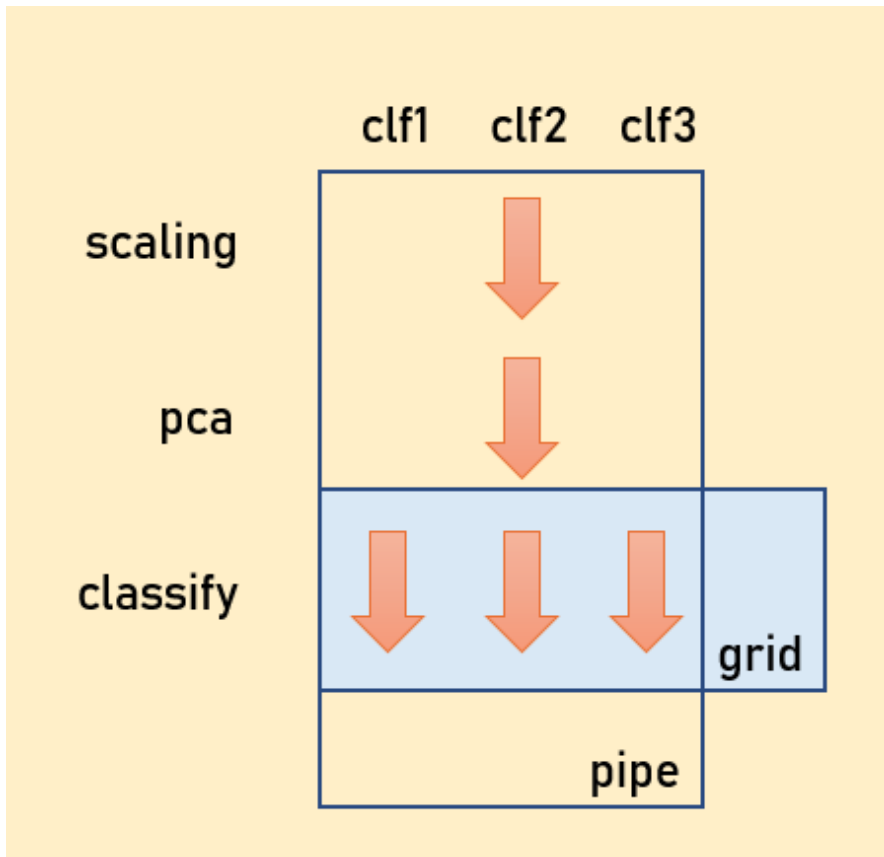


EXPERIMENT 1

- Try modifying (Face PCA)
 - Put into pipeline
 - The number of component $\rightarrow$ PCA(n_components=150)
 - Add LDA in between ***PCA and logistic regression***
- Any changes?
- You can plot in 2D projection for LDA? (optional)



HOW TO SEARCH FOR THE BEST MODEL?



```
pipe = Pipeline([  
    ("scaler", StandardScaler()),  
    ("pca", PCA()),  
    ("clf", LogisticRegression())])
```

Choose the best **PCA** component?

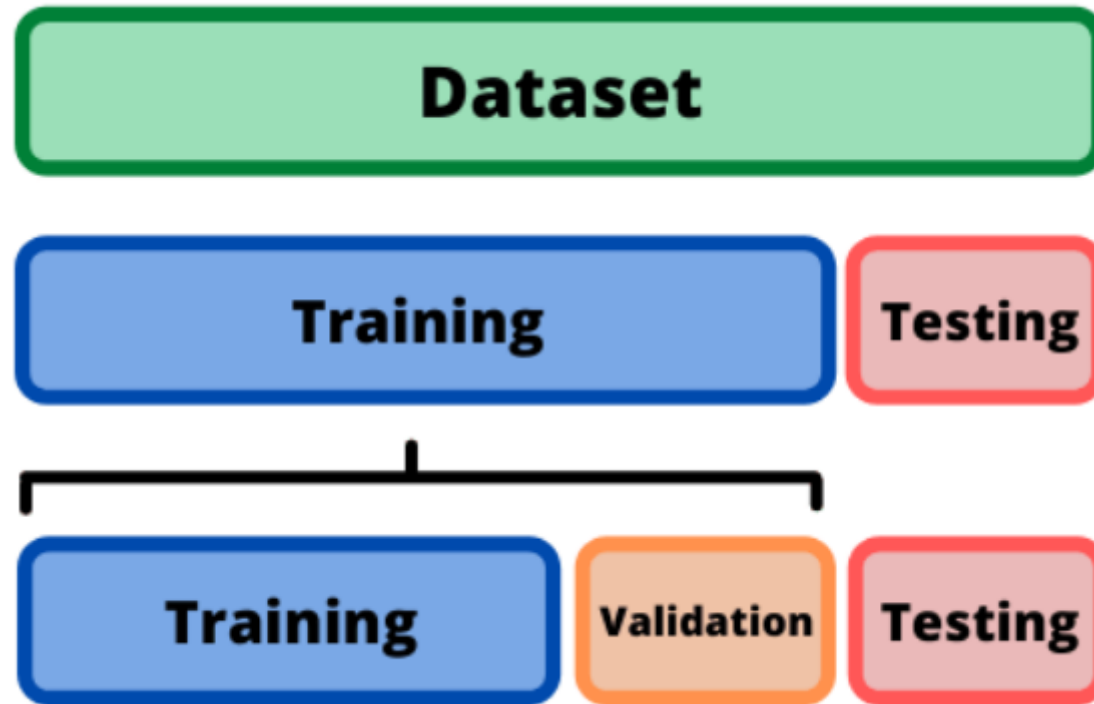


DATA SETS

- For not so small data set, obtaining the resulting performance is usually by spitting to 3 data sets.
- **Training set**
 - For training the data, from the **selected** parameters/model
- **Validating set**
 - For finding the **RIGHT models/ Right**.
 - Model: Linear, Bayesian, NonLinear
 - Parameter: solvers, parameter in each model
- **Testing set**
 - For performance evaluation from the unseen data
 - The model will **no longer changed**
 - This step will only try to obtain the **ERROR**

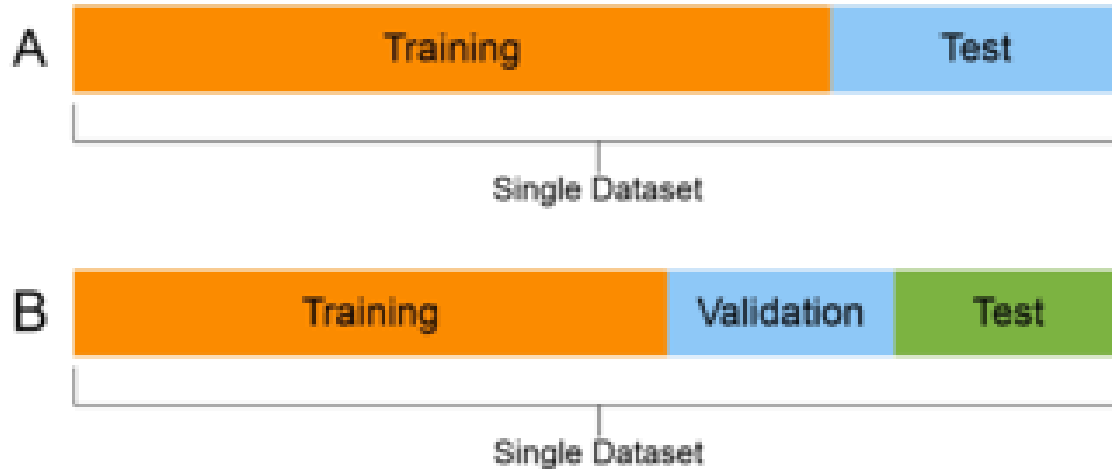


HOW DO WE SPLIT



THE HOLDOUT TECHNIQUE

- **Split dataset Directly**

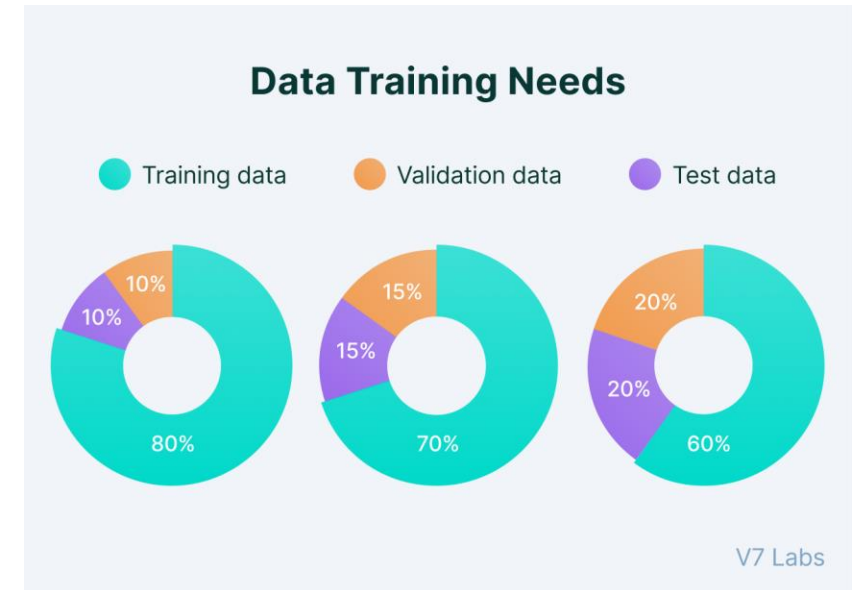


Advantage:

- Simple and easy
- Good for big enough data
- Fast

Disadvantage:

- It can be biased
- Bad splitting can create an unpleasant outcome
- May not suitable for small data set



PYTHON FOR DATA SPLITTING

- **Split the test**

```
X_train, X_test, Y_train, Y_test = model_selection.train_test_split  
    (X, Y, test_size=test_size, random_state=seed)
```

- **Split the Validation**

```
X_tr, X_val, Y_tr, Y_val = model_selection.train_test_split  
    (X_train, Y_train, test_size=test_size, random_state=seed)
```

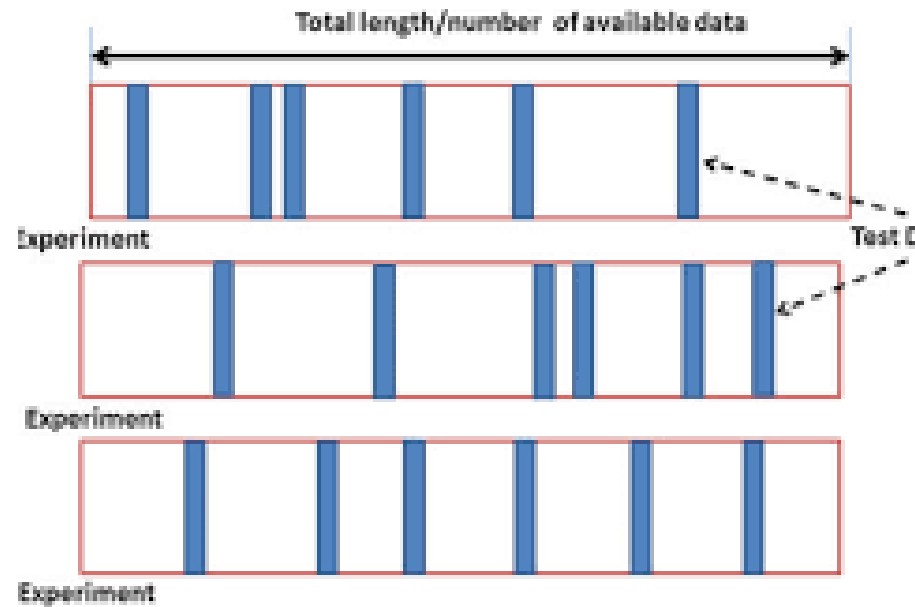


EXERCISE II

- Modify Iris data → Pipeline
- Use validation set to find the best components for iris data
 - Ranging from 1, 2, 3 (with PCA)
- Use the test set to evaluate the best result
- What happen if the random seed for data splitting is different?



CROSS VALIDATION



1.7 Data splitting in the random sub-sampling approach

- Overcome Bias in the holdout method
- Have many experiments of different sets of train/test
- Use the average for model performance



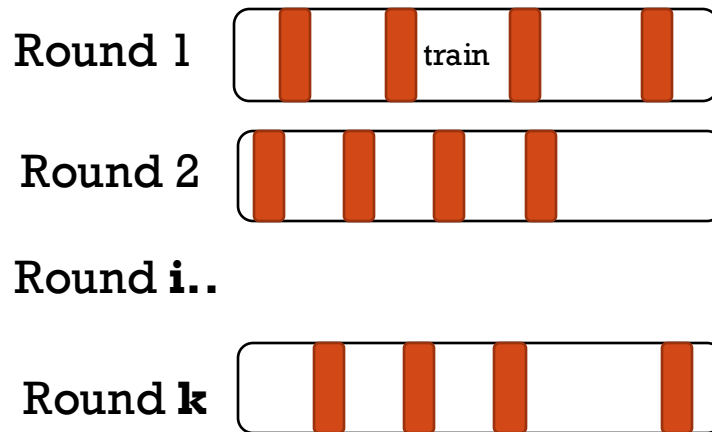
CROSS VALIDATION TECHNIQUES

- With limited number of *training* sample
- Overcome Bias in the holdout method
- Have many experiments of different sets of train/test
- Use the existing sample to find the model with the smallest error
 - Over fitting is normally an issue
 - Higher number of parameter creates more problems
 - The error rate will be too small, esp. when over fitting occur
- Cross validation is to find the best model from existing small sample.
 - To split the sample into **training** and **validation** group



CROSSVAL: RANDOM SUBSAMPLING

- **Sample fixed** number of test data from the entire data set
- **Repeat** the experiments **k**-times
- While **re-sampling** the test data



The error rate is measured by
the average of
the test errors of all **k** experiments

$$E = \frac{1}{k} \sum_i E_i$$



SKLEARN

```
▪ from sklearn.model_selection import StratifiedShuffleSplit  
sss = StratifiedShuffleSplit(n_splits=5, test_size=0.2, random_state=0)  
sss.get_n_splits(X, y)
```

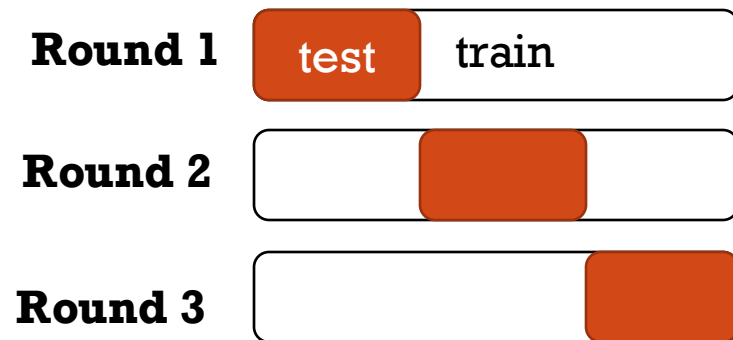
Try

```
X = np.array([[1, 2], [3, 4], [1, 2], [3, 4], [1, 2], [3, 4]])  
y = np.array([0, 0, 0, 1, 1, 1])
```



K-FOLD CROSS VALIDATION

- **Similar to Random subsampling**
 - Except **all the examples** are used in both training and testing
- **K-fold** will do the experiment **K** times
- Use **N/K** data to test and the remaining to train



3-fold cross validation

The error rate *again* use
the **average** of all **k** experiments

$$E = \frac{1}{k} \sum_i E_i$$



CROSS VALIDATION

```
from sklearn.model_selection import cross_val_score  
scores = cross_val_score(model, x_data, y_data, cv=5)
```

```
sklearn.model_selection.KFold(n_splits=5, shuffle=False, random_state=None)
```

MORE API

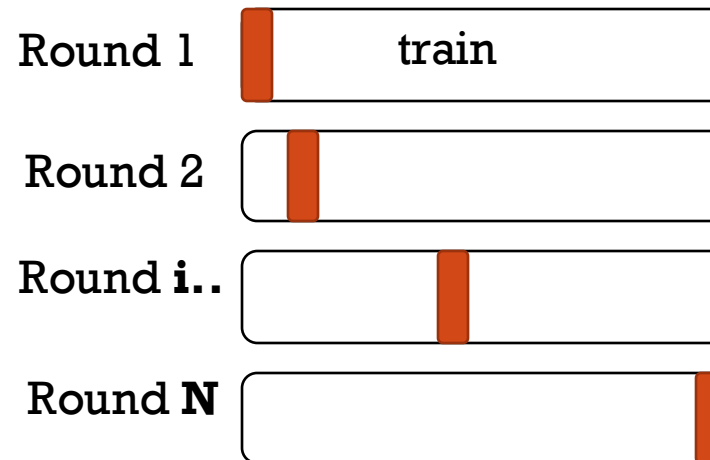
[sklearn.model_selection.KFold\(\) API](#)

[sklearn.model_selection: Model Selection API](#)



LEAVE-ONE-OUT CROSS VALIDATION

- A specific case of K-Fold cross validation
- K is chosen as **the total number** of examples
- For a dataset with **N** examples, perform N experiments (**K=N**)



**The error rate all
k experiments**

$$E = \frac{1}{k} \sum_i E_i$$



QUESTIONS?

- Try with cross validation



NUMBER OF K'S

Large K	Small K
Many experiments	Few experiments
The variance of all error can be large	Low variance tendency since number of K is small
Time consuming	Less time consuming
May have lower bias	Bias toward a particular subset can be high

Small data set is usually using leave-one-out cross-validation



BIAS-VARIANCE DECOMPOSITION

- **Variance**

- Large variety of output(performance)
- It can occur
- May happen with many **K**
- It may indicate **overfitting**

- **High Bias** (or error)

- **It may be underfitting**
- Lower variety of training
- Models bias on some particular data set
- Small **K**



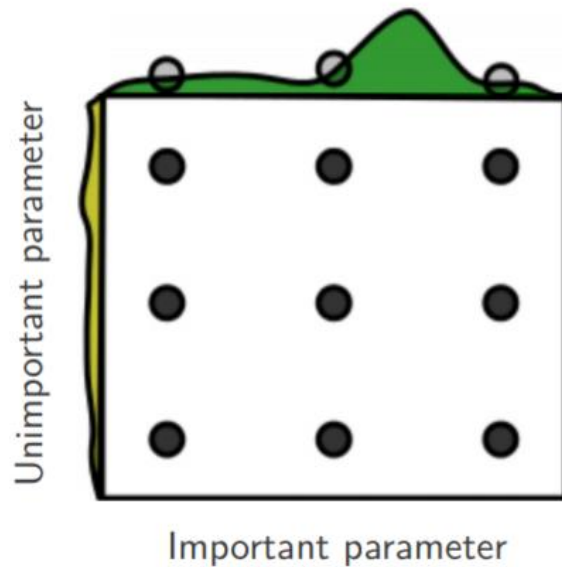
SEARCH FOR THE BEST PARAMETER

- GridsearchCV (
 - Search in grid all selected parameters
 - Set different type of hyper parameter spaces

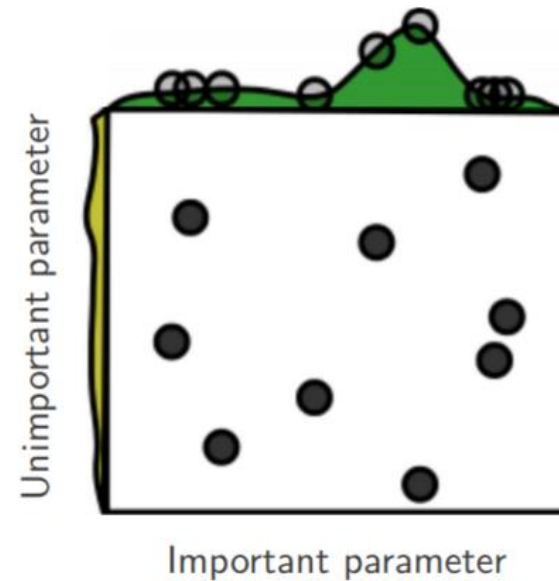


GRIDSEARCH / RANDOMSEARCH

Grid Layout



Random Layout



HOW DO WE CHOOSE?

Grid Search

```
grid_search = GridSearchCV(clf, param_grid=param_grid)
start = time() # use time to check
grid_search.fit(X, y)
print( "GridSearchCV took %.2f seconds for %d candidate parameter settings." % (time() - start,
len(grid_search.cv_results_["params"])) )
report(grid_search.cv_results_)
```

▪ Randomize Search

```
n_iter_search = 15 random_search = RandomizedSearchCV( clf, param_distributions=param_dist,
n_iter=n_iter_search )
```



RESULTS

- **Best parameter sets**
- `grid_result.best_estimator_`
- `grid_result.best_score_`

All results

- `grid_result.cv_results_`
- `sorted(result.cv_results_.keys())`



EXPERIMENTS/ ASSIGNMENT

→ Split train 75:25

Do the validation of 5-fold.

→ Find the best parameter sets

→ Show the confusion matrix

- Experiment with digit.csv to find the best parameters/setting and the best outcome
- Experiment with the face pictures best parameters and the best outcome

